

Programm: Virtual memory

```
prane_partheev1@PARTHEEV: X + v
GNU nano 8.7.1
#include <stdio.h>

int main() {
    int x = 10;

    printf("Value of x = %d\n", x);
    printf("Virtual address of x = %p\n", (void *)&x);

    return 0;
}
```

OUTPUT:

```
prane_partheev1@PARTHEEV:~$ nano practical.c
prane_partheev1@PARTHEEV:~$ gcc practical.c -o practical
prane_partheev1@PARTHEEV:~$ ./practical
Value of x = 10
Virtual address of x = 0x7ffd8ca6bfb4
prane_partheev1@PARTHEEV:~$ |
```

Programm: Demand paging

```
prane_partheev1@PARTHEEV: X + v
GNU nano 8.7.1
#include <stdio.h>

int main() {

    int page_table[4] = {1, -1, 3, -1};

    int page;

    printf("Enter page number (0-3): ");
    scanf("%d", &page);

    if (page_table[page] == -1) {
        printf("Page Fault!\n");
        printf("Page must be loaded into memory.\n");
    }
    else {
        printf("Page is already in memory.\n");
        printf("Physical frame = %d\n", page_table[page]);
    }

    return 0;
}
```

OUTPUT:

```
prane_partheev1@PARTHEEV:~$ nano demandPaging.c
prane_partheev1@PARTHEEV:~$ gcc demandPaging.c -o demandPaging
prane_partheev1@PARTHEEV:~$ ./demandPaging
-bash: ./demandPaging: No such file or directory
prane_partheev1@PARTHEEV:~$ ./demandPaging
Enter page number (0-3): 2
Page is already in memory.
Physical frame = 3
```

Programm: Process Spaces

```
prane_partheev1@PARTHEEV: ~ + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>

int global = 10;
int bss_variable;

int main() {
    int local = 20;

    int *heap = malloc(sizeof(int));
    *heap = 30;

    printf("Code address : %p\n", (void *)main);
    printf("Data address : %p\n", (void *)&global);
    printf("BSS address : %p\n", (void *)&bss_variable);
    printf("Heap address : %p\n", (void *)heap);
    printf("Stack address : %p\n", (void *)&local);

    free(heap);

    return 0;
}
```

OUTPUT:

```
prane_partheev1@PARTHEEV:~$ nano processSpaces.c
prane_partheev1@PARTHEEV:~$ gcc processSpaces.c -o processSpaces
prane_partheev1@PARTHEEV:~$ ./processSpaces
Code address : 0x555af872d1a9
Data address : 0x555af8730010
BSS address : 0x555af8730018
Heap address : 0x555b05081010
Stack address : 0x7ffc4d97af2c
```

Programm: Address Translation

```
prane_partheev1@PARTHEEV: × + v
GNU nano 8.7.1
#include <stdio.h>

int main() {
    int page_table[] = {3, 5, 1, 7};
    int page_size = 100;

    int virtual_address = 250;

    int page = virtual_address / page_size;
    int offset = virtual_address % page_size;

    int frame = page_table[page];

    int physical_address = frame * page_size + offset;

    printf("Virtual Address = %d\n", virtual_address);
    printf("Page Number = %d\n", page);
    printf("Offset = %d\n", offset);
    printf("Frame Number = %d\n", frame);
    printf("Physical Address = %d\n", physical_address);

    return 0;
}
```

OUTPUT:

```
prane_partheev1@PARTHEEV:~$ nano addressTranslation.c
prane_partheev1@PARTHEEV:~$ gcc addressTranslation.c -o addressTranslation
prane_partheev1@PARTHEEV:~$ ./addressTranslation
Virtual Address = 250
Page Number = 2
Offset = 50
Frame Number = 1
Physical Address = 150
```

Programm: Memory allocation

```

prane_partheev1@PARTHEEV:  ×  +  ▾
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>

int main() {

    int *p;

    p = malloc(5 * sizeof(int));

    if (p == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        p[i] = (i + 1) * 10;
    }

    for (int i = 0; i < 5; i++) {
        printf("%d ", p[i]);
    }

    printf("\n");

    free(p);

    return 0;
}

```

OUTPUT:

```

prane_partheev1@PARTHEEV:~$ nano memoryAllocation.c
prane_partheev1@PARTHEEV:~$ gcc memoryAllocation.c -o memoryAllocation
prane_partheev1@PARTHEEV:~$ ./memoryAllocation
10 20 30 40 50

```

Programm: Segmentation fault

```

prane_partheev1@PARTHEEV:  ×  +  ▾
GNU nano 8.7.1
#include <stdio.h>

int main() {

    int *p = NULL;

    printf("Before accessing memory\n");

    *p = 100;

    printf("After accessing memory\n");

    return 0;
}

```

OUTPUT:

```
prane_partheev1@PARTHEEV:~$ nano SegmentaionFault.c
prane_partheev1@PARTHEEV:~$ gcc SegmentaionFault.c -o ^C
prane_partheev1@PARTHEEV:~$ nano SegmentaionFault.c
prane_partheev1@PARTHEEV:~$ gcc SegmentaionFault.c -o SegmentaionFault
prane_partheev1@PARTHEEV:~$ ./SegmentaionFault
Before accessing memory
Segmentation fault ./SegmentaionFault
```